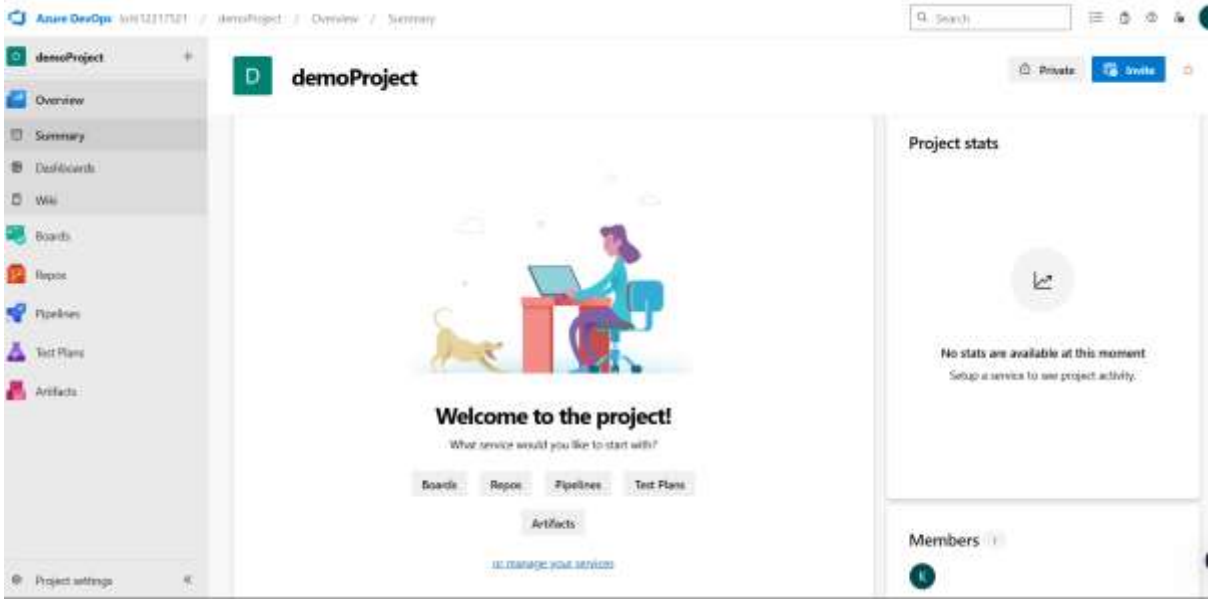
Name: Kriti
College: Lovely Professional University
Week7 Assignment
Email: kritijulia@gmail.com
Student_Id: CT_CSI_DV_5113

Q1. Create a project with different user groups and implement group policies.

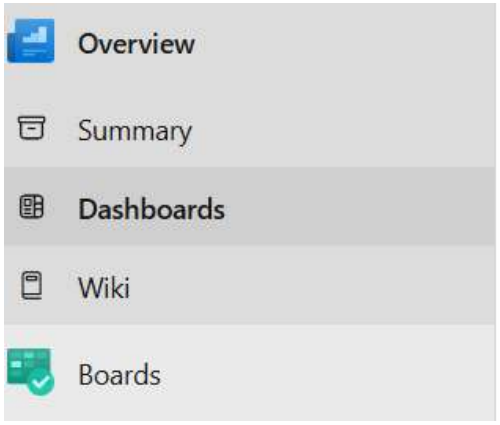
Answer:

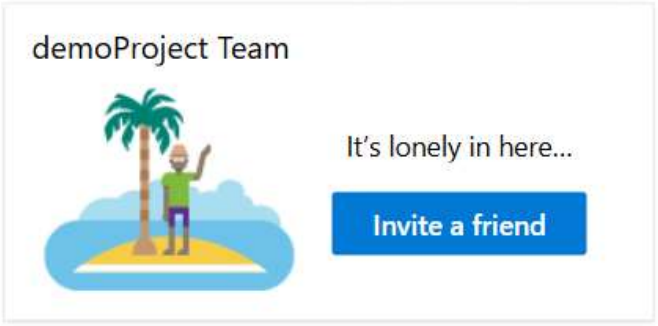
Step1: create an azure devops account and make an organization in organization make a project by giving name and description of the project you can also choose serves like agile, scrum etc.. This is a overview of my demoproject.

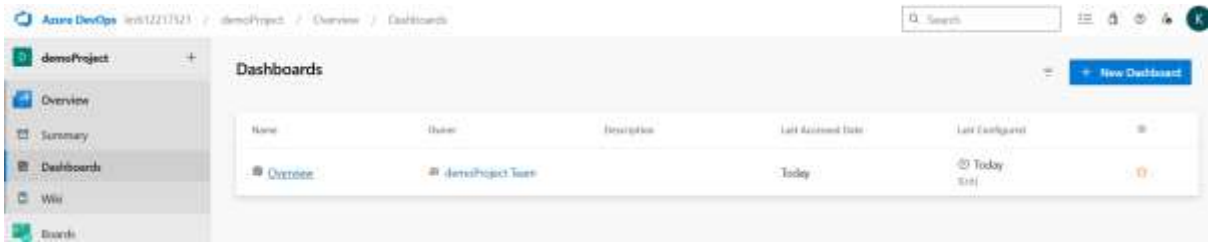


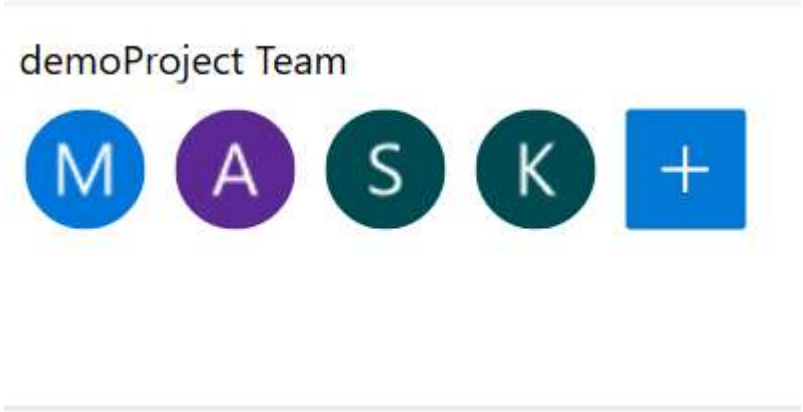
Step2: Now I will go to dashboard and add team member widget to manage the team members and

invite new members. By clicking on + you can add new members also.





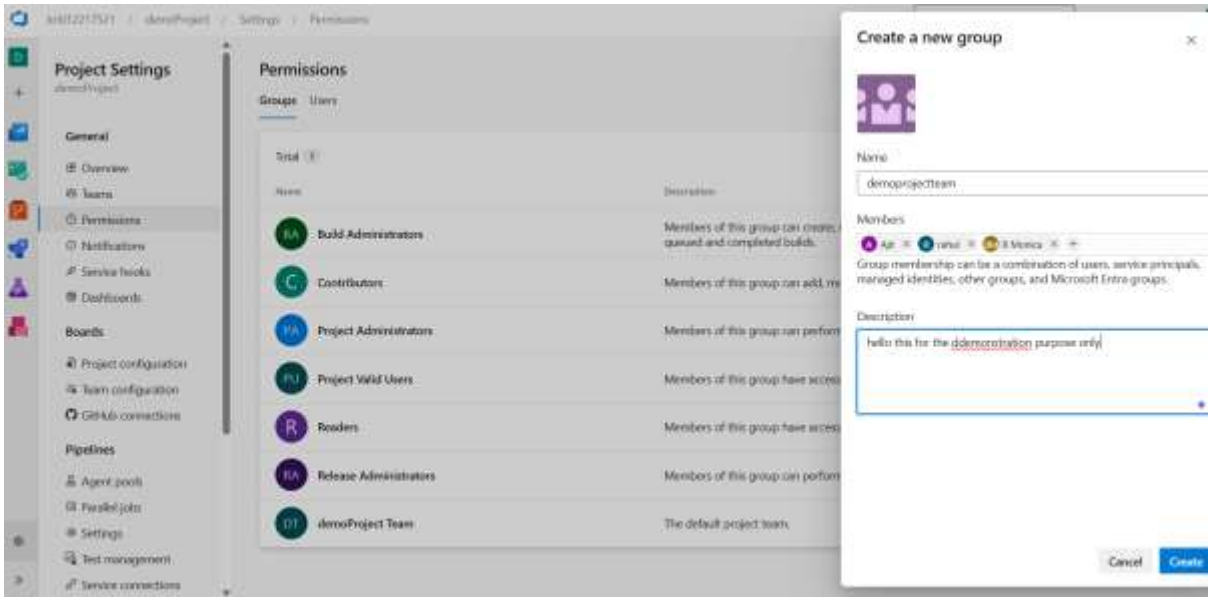




Step 3: **Create User Groups**

I will Navigate to Project Settings → Permissions

1. Click **New Group** for each role:
    - Group Name: Project Admins
    - Group Name: Developers
    - Group Name: Testers
2. **Add Users** to each group:
    - Click the group → **Members** → **Add**
    - Enter user emails to invite and assign.



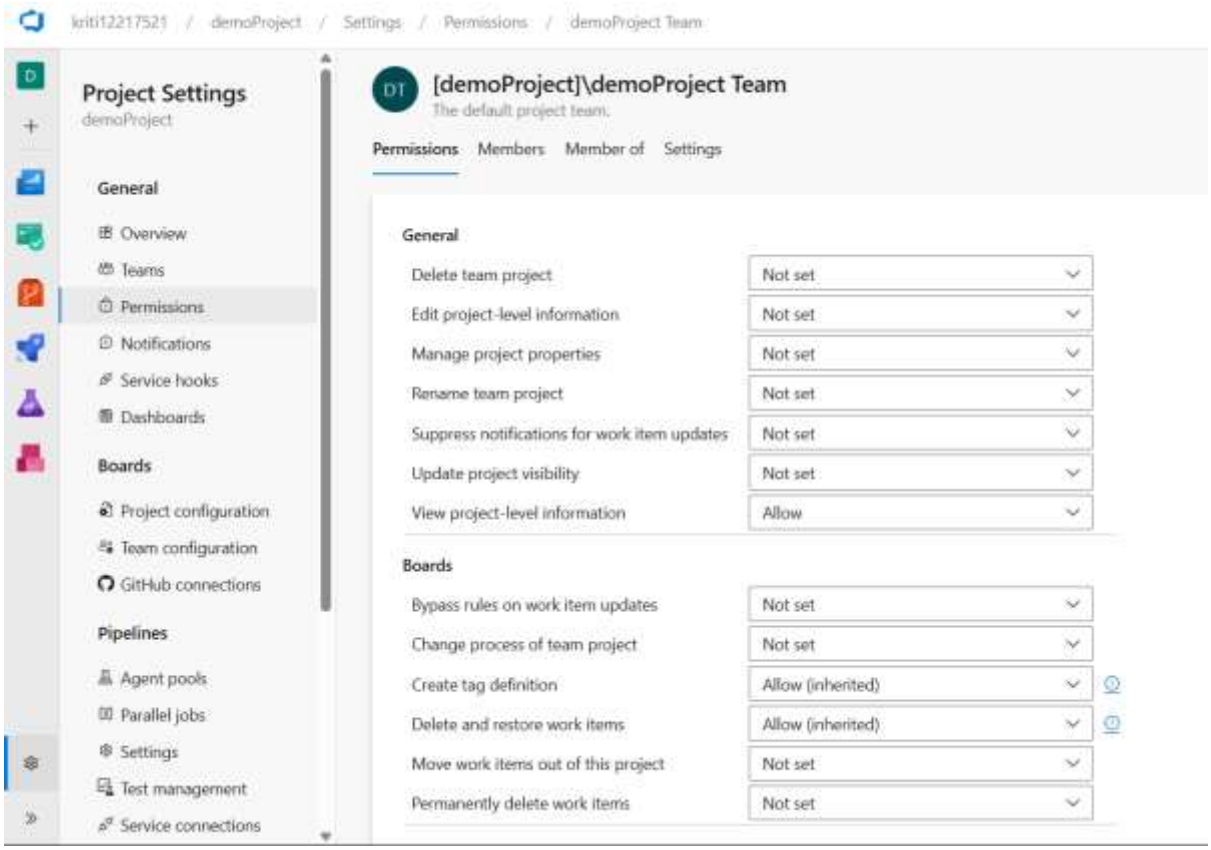
Step 4: Assign Permissions to Groups

Go to:

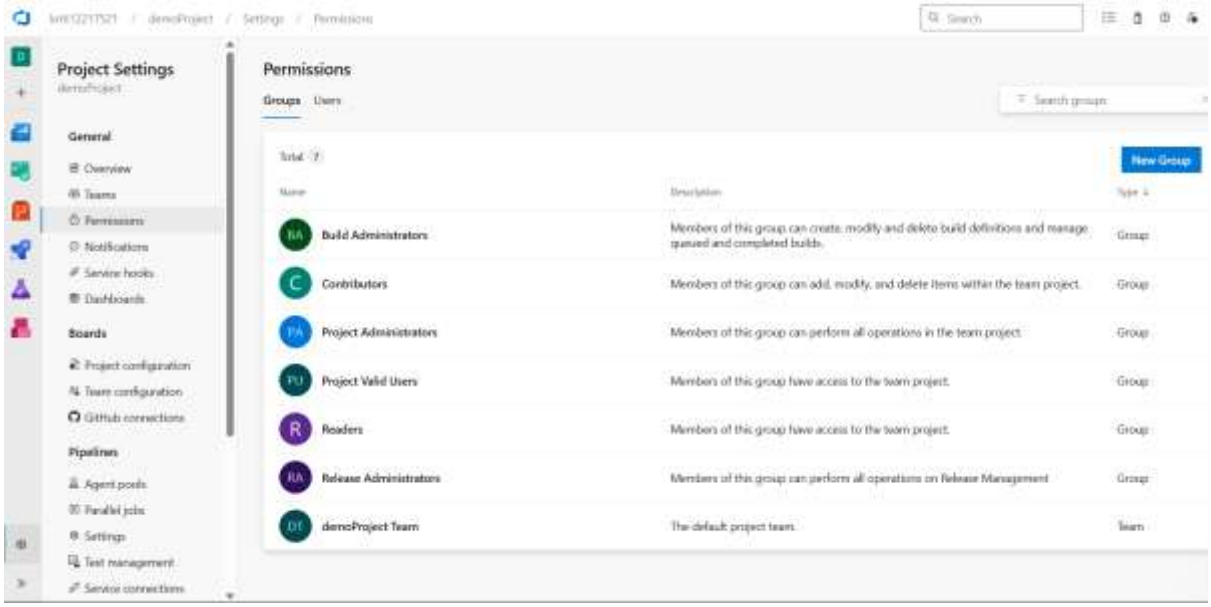
Project Settings → Permissions → [Select Group] → Permissions

Example settings:

- **Project Admins**
  - Edit project-level information: **Allow**
  - Administer build permissions: **Allow**
  - Manage permissions: **Allow**
- **Developers**
  - Contribute to repositories: **Allow**
  - Queue builds: **Allow**
  - Create branches and tags: **Allow**
  - Delete repository: **Deny**
- **Testers**
  - View build pipeline: **Allow**
  - View releases: **Allow**
  - Contribute to work items: **Allow**
  - Edit code: **Deny**



these all are the by default group make by azure devops



We can manage and allow user and group contribute to the project and other stuff.

We can do all the above step using command line also for that you have to install azure devops cli extension also.

Step 5: Implement Branch Policies (Important)

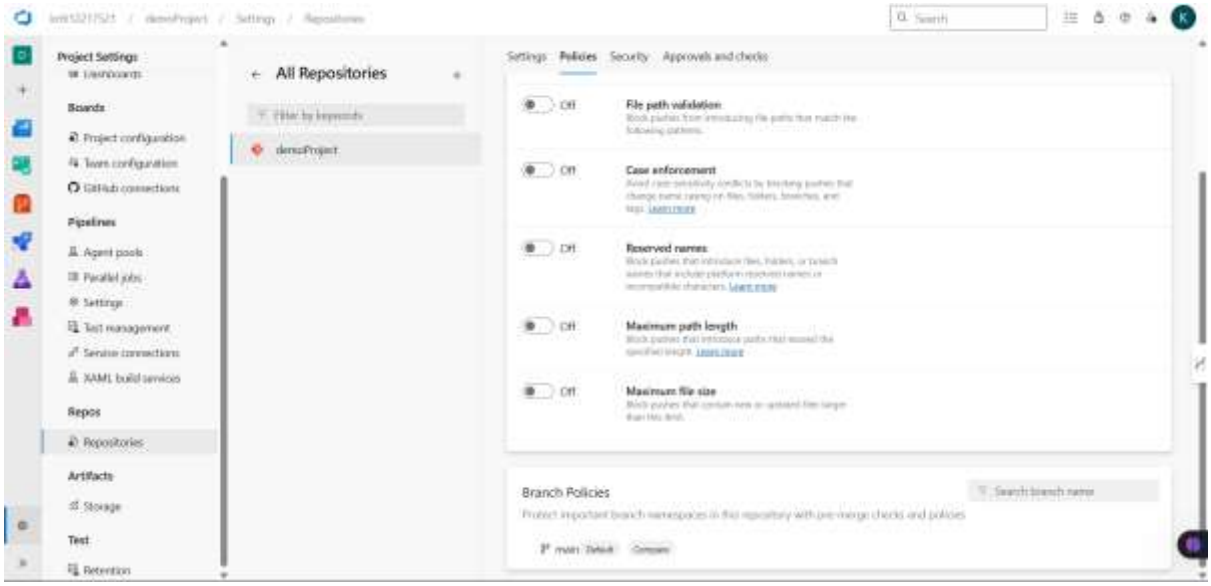
To enforce coding standards and code reviews:

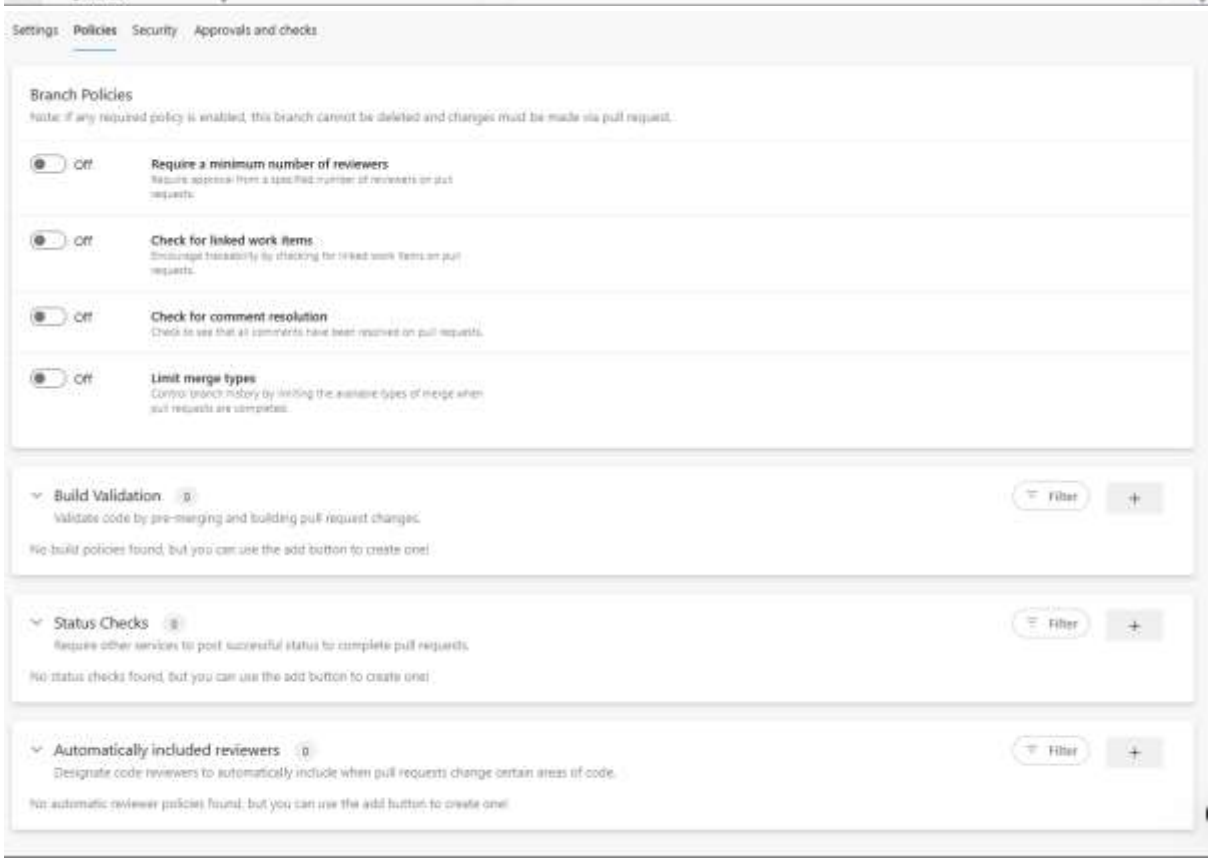
Go to:

Repos → Branches → Select Branch (e.g., main) → ... → Branch Policies

1. **Minimum number of reviewers** → 2
2. **Check for linked work items** → Enabled
3. **Limit merge types** → Squash or Rebase
4. **Require successful build** → Select a pipeline
5. **Block push if checks fail** → Enabled

Apply to Developers group only (Admins can bypass).





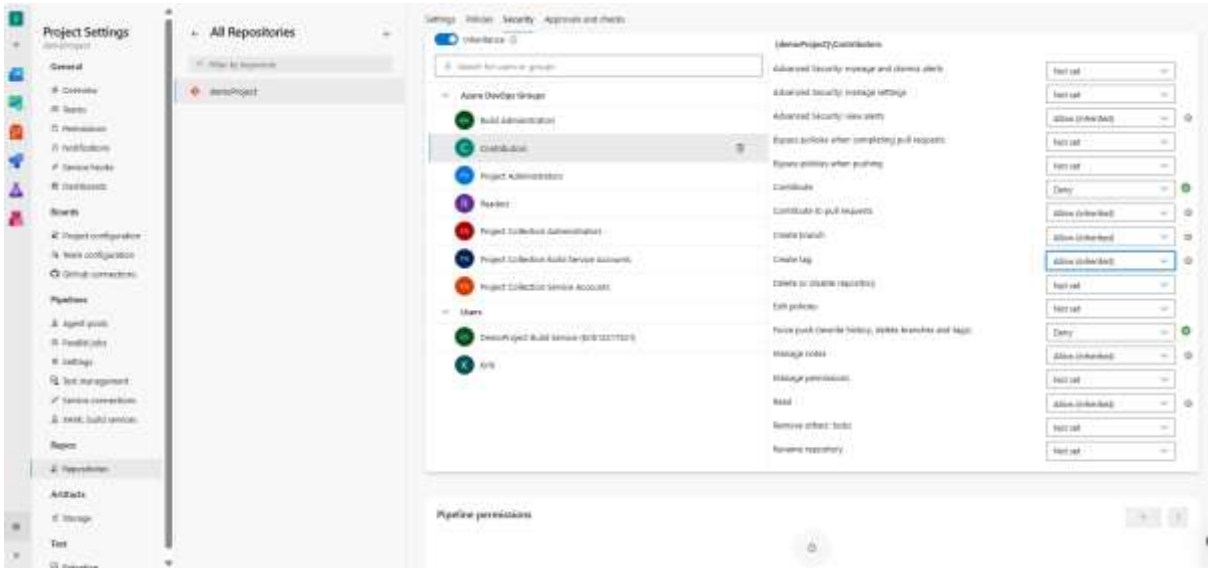
Q2. Apply branch policies such that only the project administrator can access the master branch; contributors cannot.

Step1: Go to Repos → Branches → Select Branch (e.g., main) → ... → Branch Policies
Step 2: Set Permissions for Contributors

In the Branch security dialog:

1. Select the Contributors group.
2. Set these permissions:
    - Contribute → Deny
    - Force push (rewrite history and delete branches) → Deny
    - Create tag → Optional (Deny if needed)
    - Read → Allow (keep if they should be able to clone/view)

This will block all write access (push/merge) to the master branch for contributors.

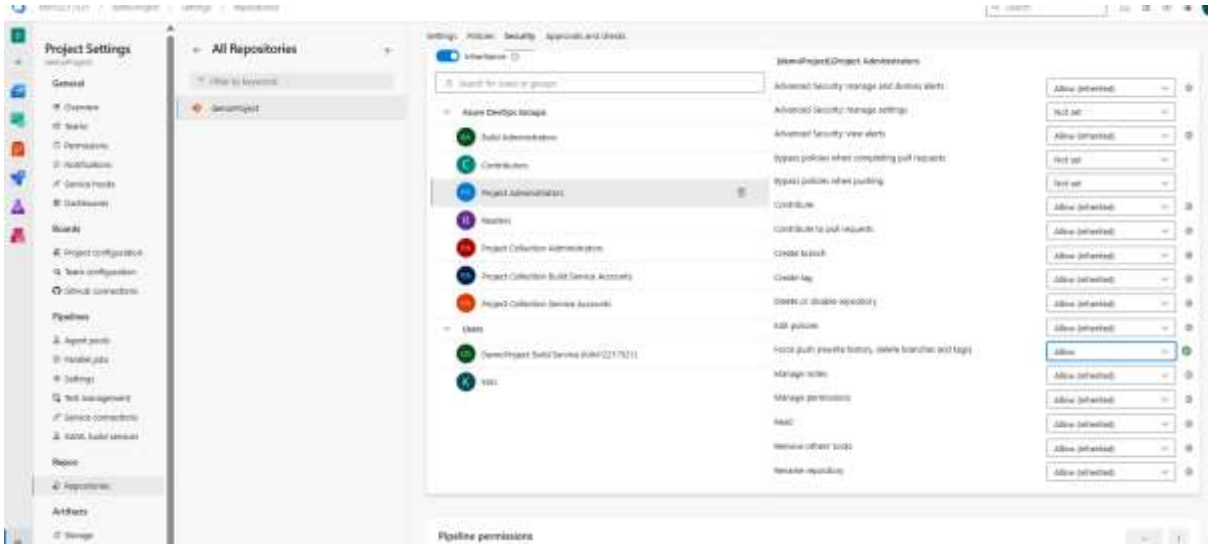


Step 3: Set Permissions for Project Administrators

1. Select the Project Administrators group.
2. Ensure these are set:
    - Contribute → Allow
    - Force push → Allow (optional)
    - Read → Allow

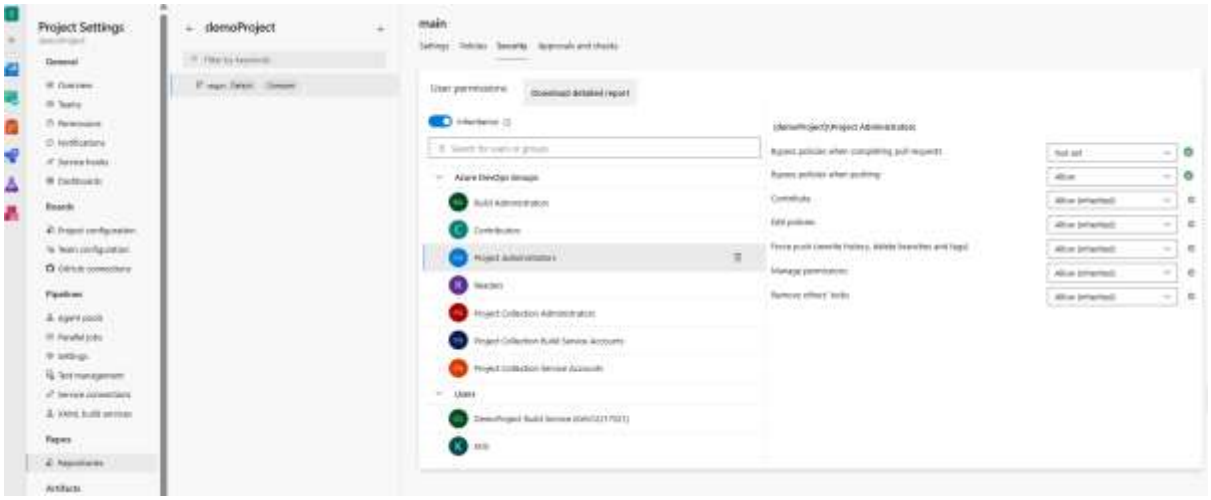
Step 4: Optional – Add Branch Policies for Visibility

Although we've denied access to contributors, you can also reinforce this with policies.



Navigate to:
Repos → Branches → master → ... → Branch policies

1. Set policies only useful for Admins:
    - Bypass policies when pushing → Allow only for Project Administrators
    - Require a minimum number of reviewers → Not required (since contributors can't push)

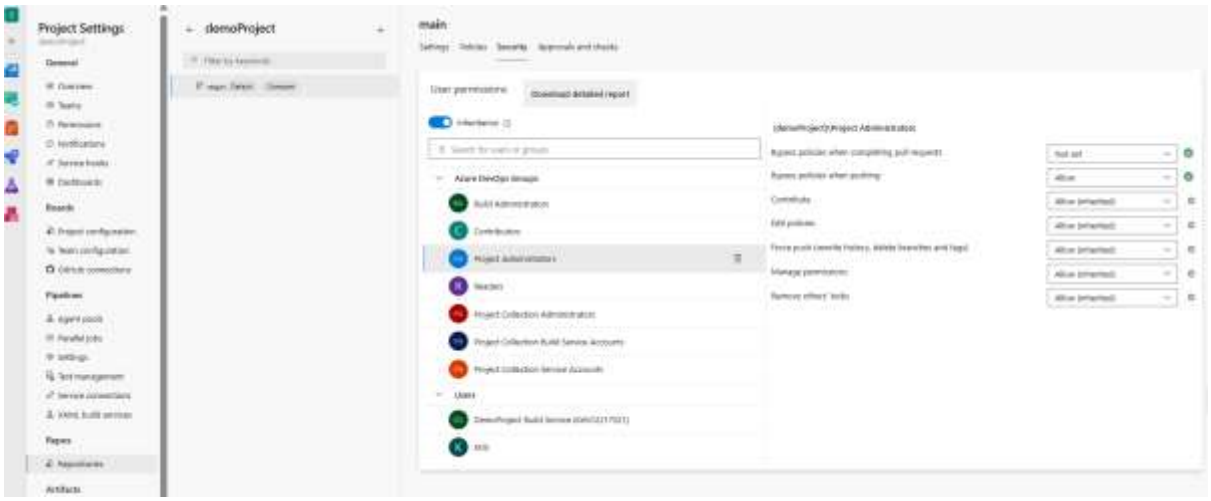


Q3. Apply branch security and locks.

Contributor Permission	Value
Contribute	Deny
Bypass policies when pushing	Deny
Bypass policies when completing PR	Deny
Force push (rewrite history)	Deny
Edit policies	Deny

Administrator group Permission	Value
Contribute	Allow
Bypass policies	Allow
Force push	Allow (optional)
Edit policies	Allow

Go to Repos → Branches → Select Branch (e.g., main) → ... → **Branch security**.



Apply Branch Lock

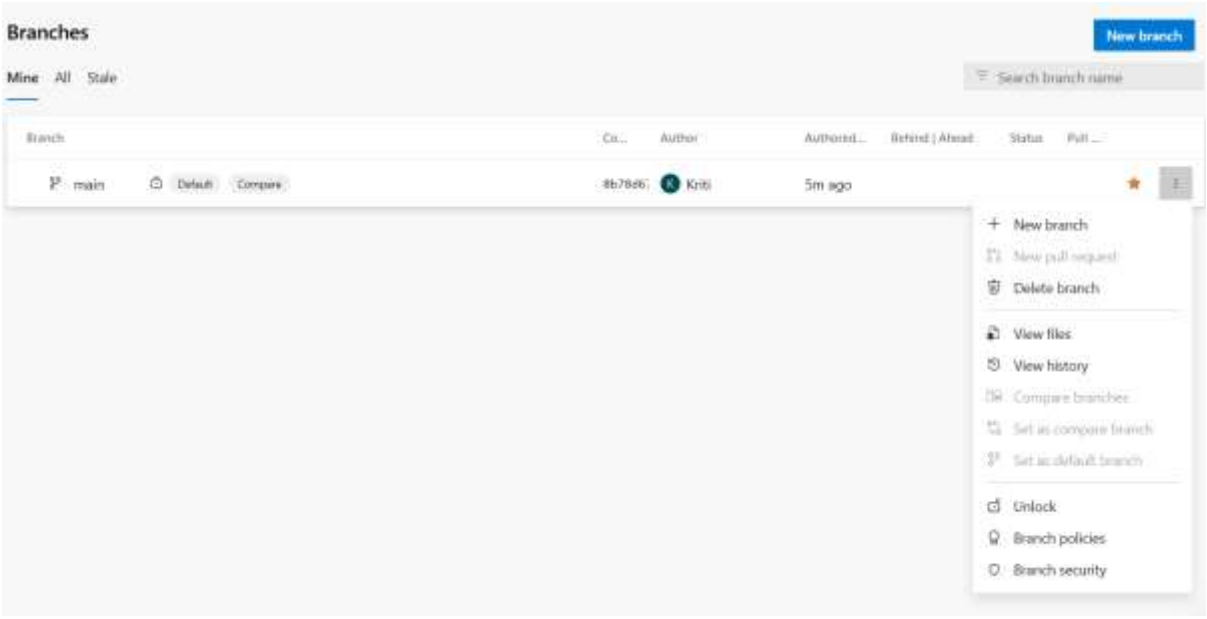
A branch lock **prevents any changes** — no push, merge, or deletion, even by admins unless it's unlocked.

How to apply:

1. Go to Repos → Branches.
2. Click the ... next to master.
3. Select **Lock**.

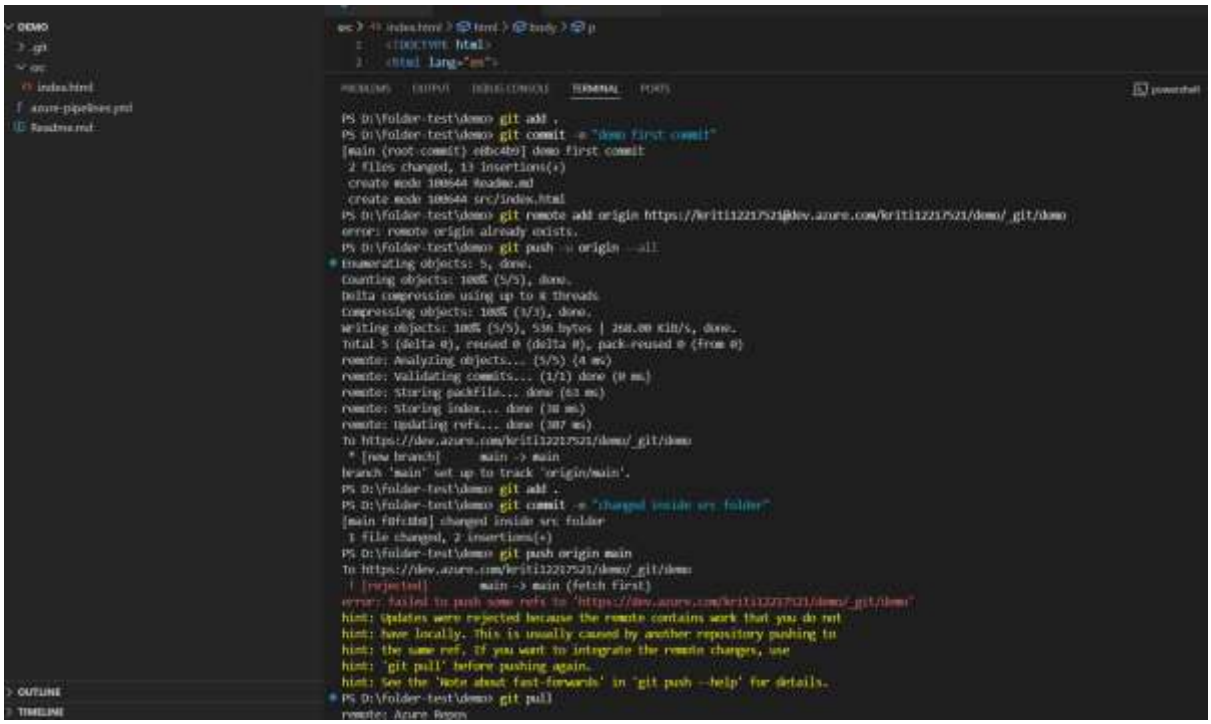
🔒 Once locked:

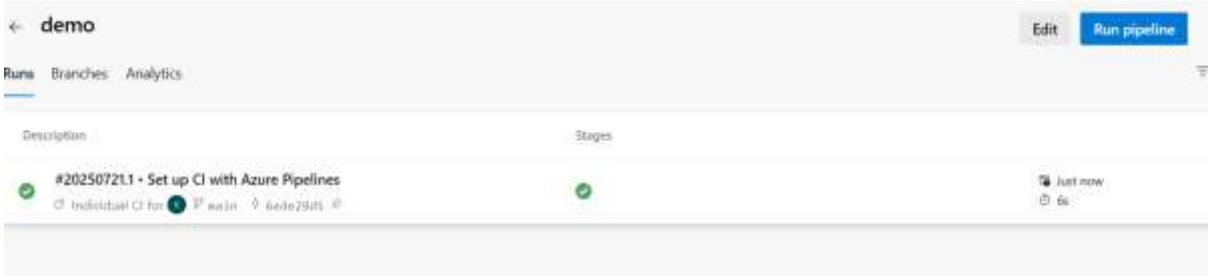
- No one (even admins) can push or complete PRs to it.
- You'll see a lock icon next to the branch name.



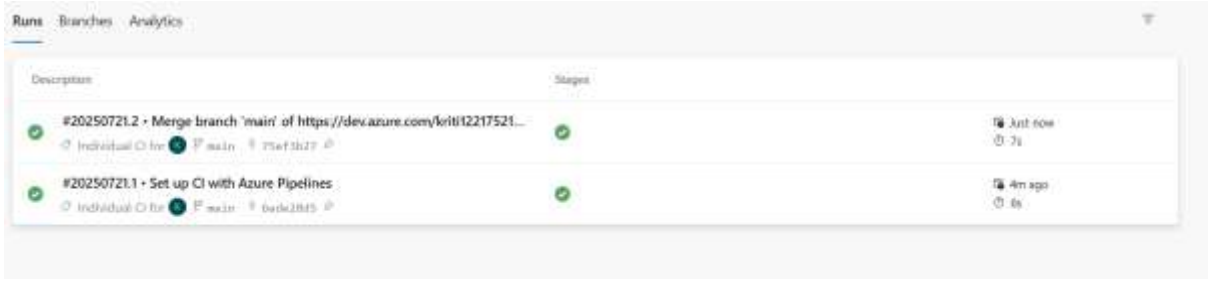
Q4. Apply branch filters and path filters.

Create a new demo project then add a yaml file as shown which will trigger only when the push is to the main branch and files under src/ folder are modified





After new commit the demo pipeline.



After changing some content inside src folder it is updated and new pipeline generated

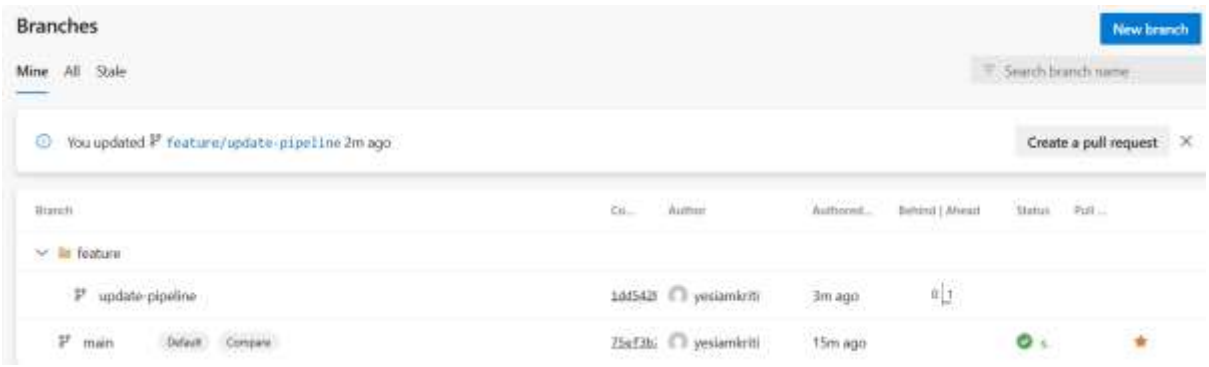
Q5. Apply a pull request.

A **Pull Request** allows you to:

- Propose changes from one branch (e.g., feature/login) into another (e.g., main)
- Review and approve code before merging
- Trigger your pipeline if pr: filters are defined

Here I am creating a new branch feature/ update-pipeline and then creating a index.js file inside src and push it to azure git.

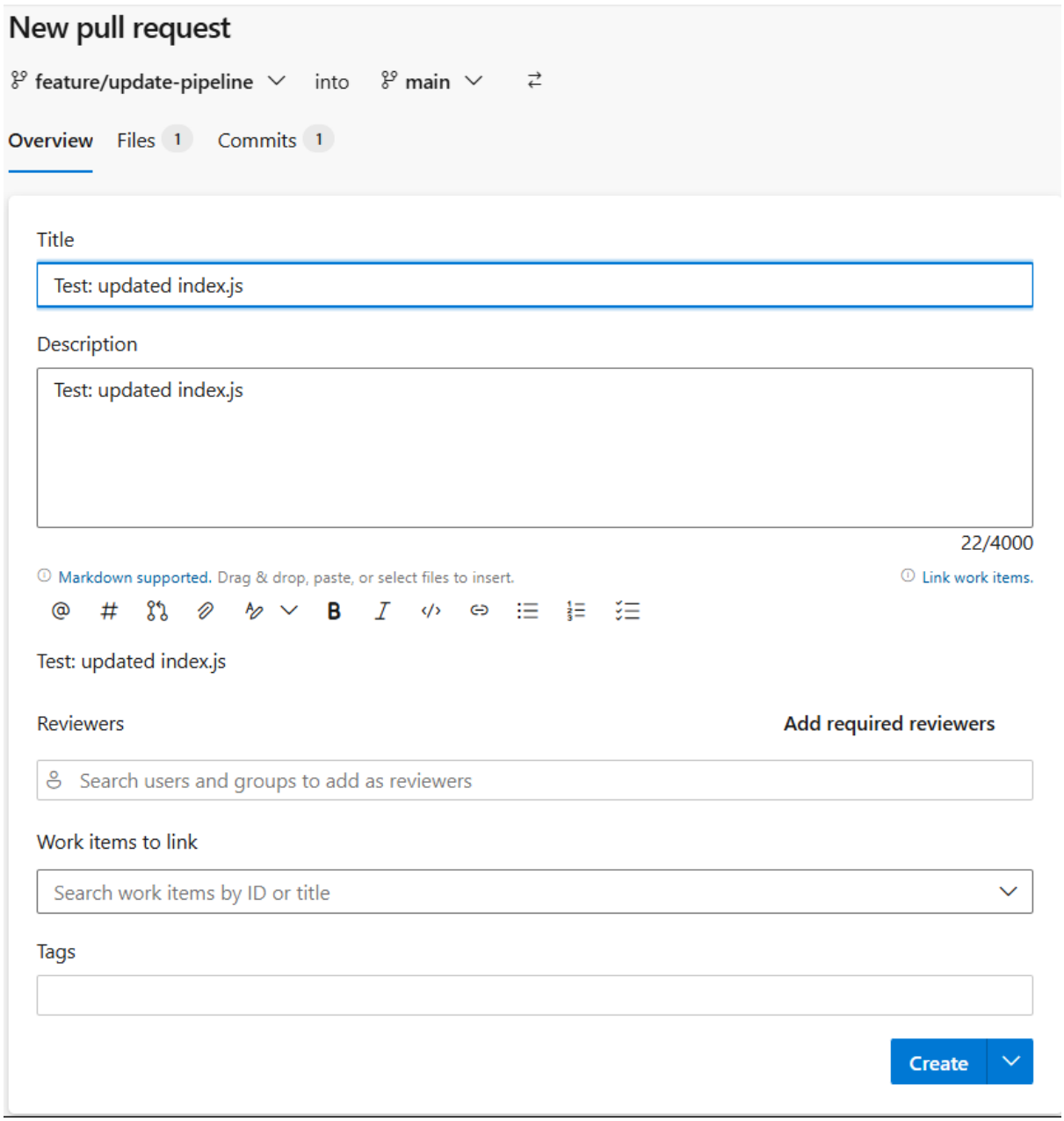
```
PS D:\folder-test\demo> git checkout -b feature/update-pipeline
Switched to a new branch 'feature/update-pipeline'
PS D:\folder-test\demo> git add .
PS D:\folder-test\demo> git commit -m "Test: updated index.js"
[feature/update-pipeline 1dd542b] Test: updated index.js
 1 file changed, 1 insertion(+)
 create mode 100644 src/index.js
PS D:\folder-test\demo> git push origin feature/update-pipeline
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 399 bytes | 199.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Analyzing objects... (4/4) (6 ms)
remote: Validating commits... (1/1) done (1 ms)
remote: Storing packfile... done (33 ms)
remote: Storing index... done (40 ms)
remote: Updating refs... done (143 ms)
To https://dev.azure.com/kriti12217521/demo/_git/demo
 * [new branch]      feature/update-pipeline -> feature/update-pipeline
PS D:\folder-test\demo>
```

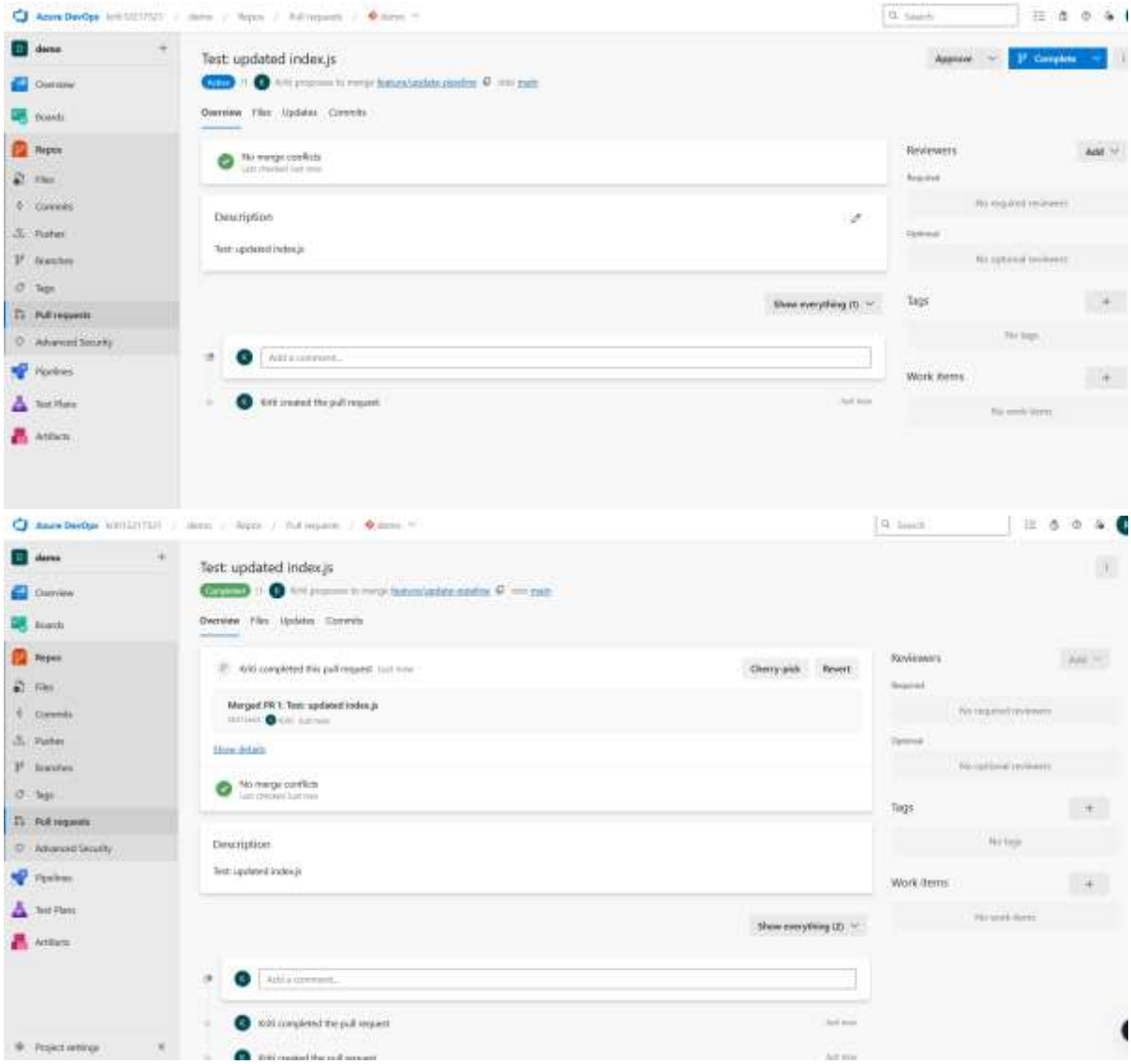


Here a option create pull request option is available we click over and our task is done.

step 2: Create the Pull Request

1. Go to **Azure DevOps → Repos → Branches**
2. You'll see your new branch (feature/update-pipeline)
3. Click **"Create a pull request"**
4. Set:
    - **Source branch**: feature/update-pipeline
    - **Target branch**: main
5. Add title & description (optional)
6. Click **Create**


New pull request
feature/update-pipeline
into
main
Overview
Files 1
Commits 1
Title
Test: updated index.js
Description
Test: updated index.js
22/4000
Markdown supported. Drag & drop, paste, or select files to insert.
Link work items.
Test: updated index.js
Reviewers
Add required reviewers
Search users and groups to add as reviewers
Work items to link
Search work items by ID or title
Tags
Create



Step 3: Add PR Trigger in Pipeline YAML

Update your .azure-pipelines.yml

```yaml
trigger:
pr:
  branches:
    include:
      - main     # Trigger only for main branch
  paths:
    include:
      - src/**   # Trigger only if files in src/ folder change

pool:
  vmImage: ubuntu-latest

steps:
  - script: echo "This runs only when changes are pushed to main branch under src/"
    displayName: 'Branch & Path Filter Demo'
```

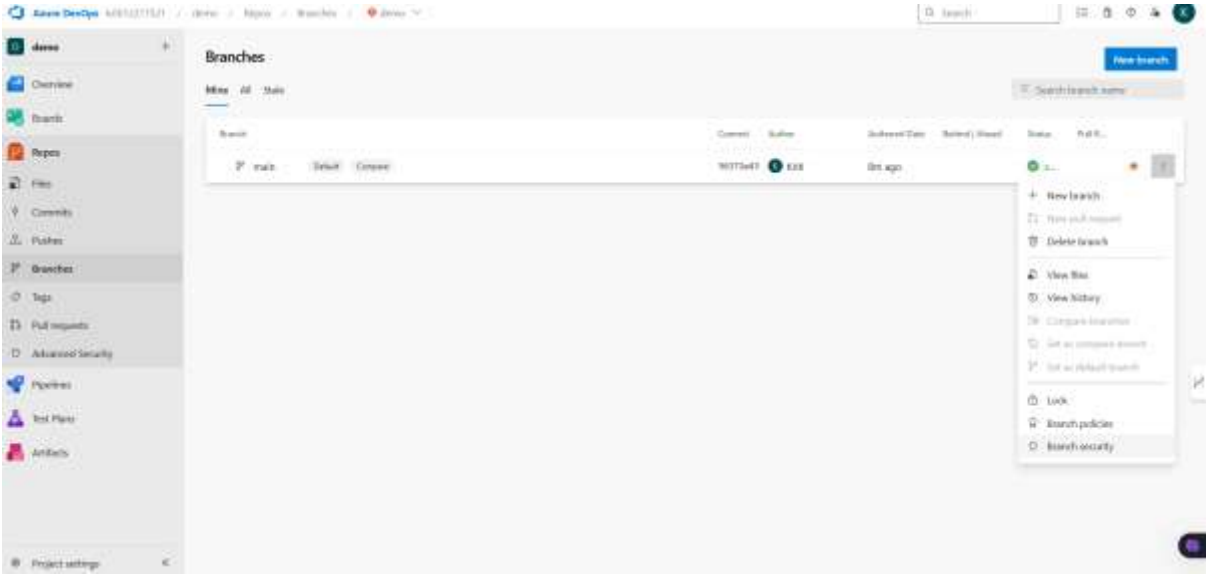
If I do this then whenever I update from feature branch then only it will update the pipeline.

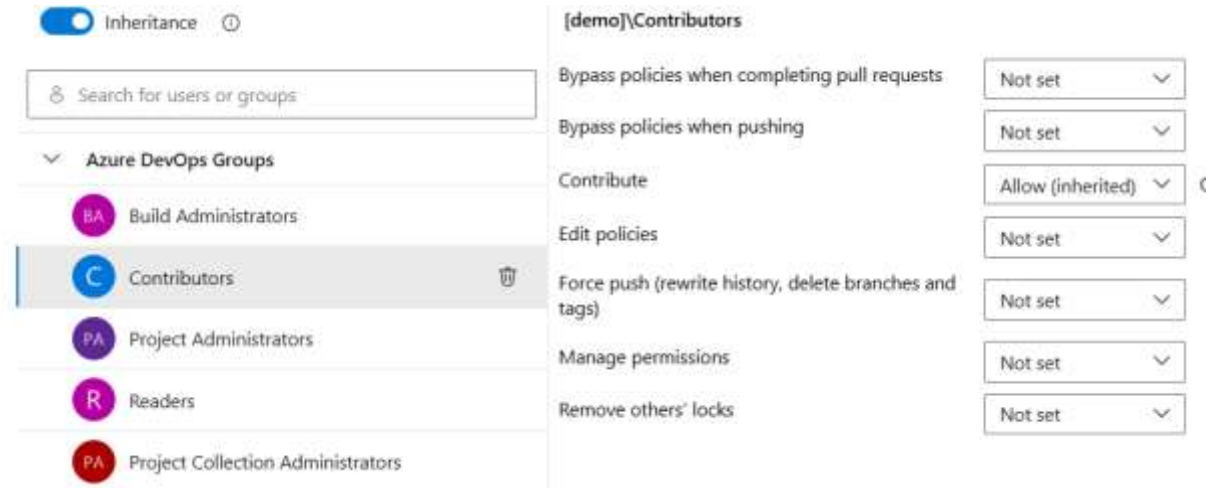
Q6. Apply branch policy, Apply branch security.

To apply first got to repo->branches->three dots you see branch policy and apply branch security option click and set permissions. Here I am applying as

Contributors Permission	Value
Contribute	Deny
Force push (rewrite history)	Deny
Bypass policies when pushing	Deny
Bypass policies when completing PRs	Deny
Edit policies	Deny

Project Administrators Permission	Value
Contribute	Allow
Bypass policies	Allow
Edit policies	Allow



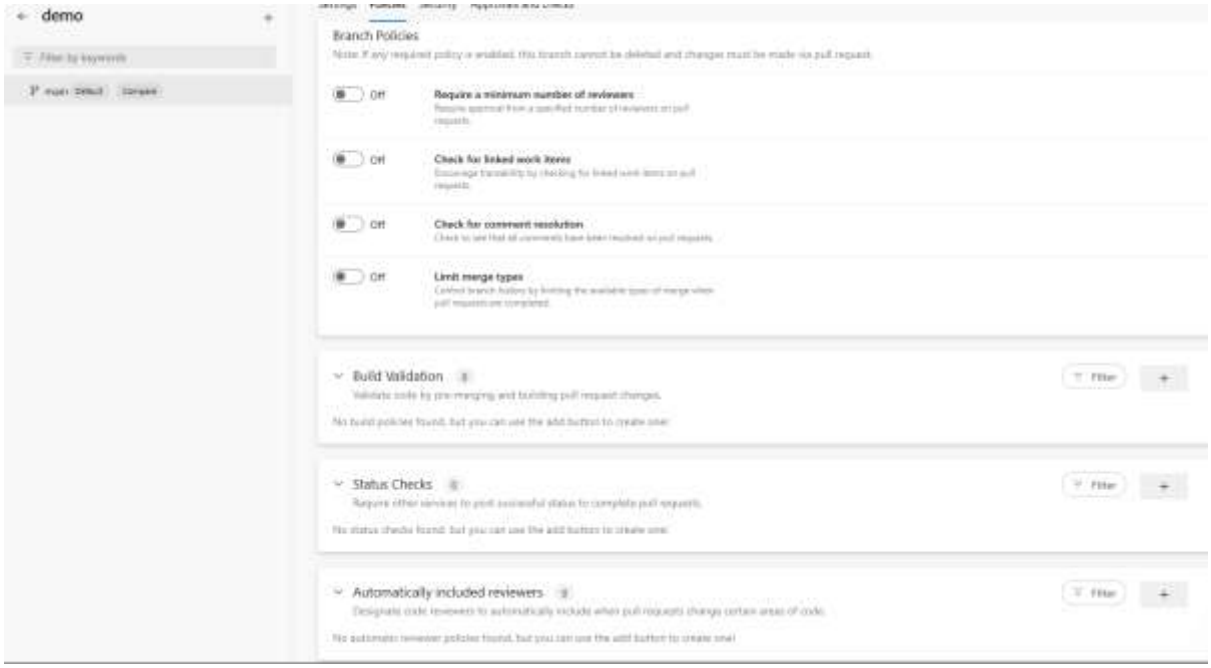


Step 2: Apply **Branch Policies**
To apply branch policies in Azure DevOps, first navigate to Repos → Branches, then click the three dots (...) next to the main branch and select Branch policies. In the

configuration panel, start by enabling "Require a minimum number of reviewers", and set the minimum to 1 or 2, depending on your team's review standards. You can also optionally add specific reviewer groups, such as Project Administrators, to ensure critical changes are reviewed by key members.

Next, enable the option to "Check for linked work items" to enforce traceability between code changes and tasks or bugs. Also, activate "Check for comment resolution" to prevent merges if there are unresolved comments in the pull request, encouraging cleaner and more accountable code reviews.

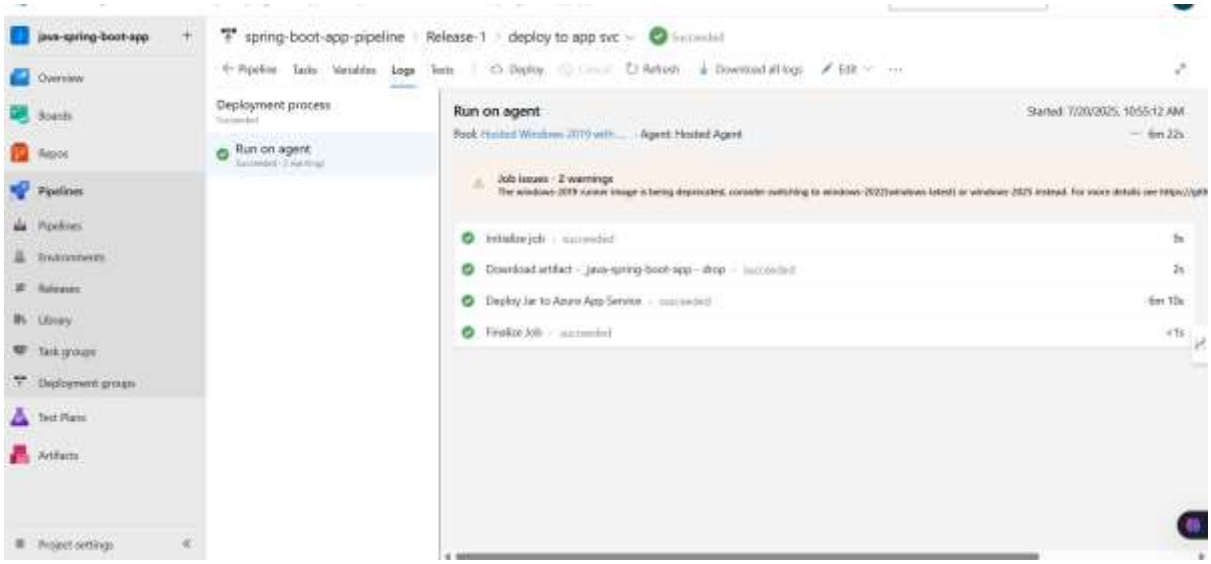
Under "Limit merge types", select allowed options such as "Squash" or "Rebase and merge" to maintain a clean and linear Git history. Lastly, for continuous integration, add a build validation policy by clicking “+ Add build policy”, selecting your YAML pipeline, and enabling the “Trigger on pull request” option. Once all configurations are complete, click Save to apply the policies to the branch.



Q7. Apply triggers in build and release.

To apply triggers in both the build and release pipelines in Azure DevOps, I started by configuring the triggers in the build pipeline using YAML. I included the trigger block to specify that the pipeline should run automatically whenever changes are pushed to the main branch, and I added the pr block to ensure the pipeline runs on pull requests targeting the main branch as well. Additionally, I used path filters to narrow down the trigger to specific folders—so the pipeline runs only when files within the src/ folder are modified. For the release pipeline, I opened the Releases section in Azure DevOps and either created a new release pipeline or selected an existing one. I then added an

artifact by choosing the source type as Build and linking it to the corresponding build pipeline. After adding the artifact, I enabled the Continuous Deployment Trigger by clicking on the lightning bolt icon and turning it on. This ensures that every time a successful build completes, a release is automatically triggered. Optionally, I also scheduled builds using the schedules block in the YAML file, where I specified a cron expression to run the build at a fixed time—for instance, every Monday at 2:00 AM. Similarly, in the release pipeline, I configured stage-level scheduled triggers by going into the stage settings and setting up a release to occur at a specific time or interval. This setup automates both the build and deployment processes, ensuring that new changes are continuously integrated and deployed without manual intervention.



Q8. Apply gates to the pipeline.

To enhance the release pipeline with quality and control, I applied gates in Azure DevOps. Gates are conditions that must be satisfied before a deployment to an environment can proceed. I started by navigating to the release pipeline and selecting the environment where I wanted to apply the gates. Under the environment's pre-deployment conditions, I enabled the **"Gates"** option. Then, I configured one or more gates such as calling a REST API, checking for work item status, or integrating with Azure Monitor alerts. For example, I added a gate to invoke a REST API that returns a health status of the system before continuing the release. I also configured a gate that queries for active bugs or open work items tied to the release—ensuring we don't deploy if there are unresolved issues. Each gate had a timeout and sampling interval, which I customized based on the expected response times and frequency of checks. Once configured, these gates help enforce quality by preventing deployment to production unless certain external checks pass, adding an additional layer of validation to the pipeline.

Q9. Apply security on branches so that contributors can create a pull request but cannot directly merge code to master.

To ensure code quality and maintain control over the main branch, I applied security settings that prevent contributors from directly pushing changes to the main (or master) branch. Instead, contributors must create a pull request (PR), which can be reviewed and approved before merging. I navigated to the **Repos > Branches** section in Azure DevOps, then selected the main branch and clicked on the three dots (...) to access **Branch Security**. In the security settings, I selected the contributor group or individual users and explicitly **denied** the "Contribute" and "Force push" permissions. At the same time, I made sure the "Create branch" and "Create pull request" permissions remained **allowed**, so they could still collaborate through PRs. This setup enforces a policy where all code changes must go through code review and meet branch policies (like required reviewers or build validations), promoting better quality and accountability before merging to the main branch.

Q10. Use work items in pipelines.

To associate work items with pipeline runs in Azure DevOps, I followed these steps:

1. Link Work Items in Commits or Pull Requests

When committing code or creating a pull request, I referenced the work item ID in the commit message or PR description using the format #<work item ID>.

Example:

git commit -m "Fix login bug #123"

This automatically links work item **#123** to the pipeline run triggered by that commit.

2. Enable Work Item Tracking in Pipeline

Azure Pipelines automatically associates work items if:

- The pipeline is linked to a Git repo hosted in Azure Repos.
- The commit message or PR contains valid work item references.

I made sure this was configured by going to:

Pipelines > Edit Pipeline > Options, and enabled:

- ✅ "Report build status"
- ✅ "Associate work items"

3. View Associated Work Items

After a pipeline run, I opened the run details. Under the **"Related"** or **"Summary"** section, I could see the linked work items (bugs, tasks, etc.) that were mentioned in commits or PRs.

Using work items in this way helps track which feature or bug fix each deployment includes, improving traceability and collaboration between developers and project managers.